

Projet 1 — Déploiement et Exploitation d'un Cluster Hadoop avec Docker

- Membres du groupe
- Introduction
- Lexique (sigles utiles)
- Partie 1 — Contexte théorique et architecture (5 pts)
 - 1. Cas d'usage Big Data (1.5 pt)
 - 2. HDFS vs système de fichiers classique (1.5 pt)
 - 3. Rôle de YARN — ResourceManager et NodeManager (2 pts)
- Méthode de travail
- Partie 2 — Déploiement et manipulation pratique (15 pts)
 - A. Mise en place de l'environnement Docker (4 pts)
 - B. Manipulation avancée sur HDFS (6 pts)
 - C. Exécution d'un job YARN (5 pts)
- Partie 3 — Administration, monitoring et analyse (10 pts)
 - 1. Interfaces web (4 pts)
 - 2. Facteur de réplication (3 pts)
 - 3. Retour d'expérience / troubleshooting (3 pts)
- Conclusion
- Annexes
 - A. Structure du code source
 - B. Liste des captures
 - C. Liens du projet

Projet 1 — Déploiement et Exploitation d'un Cluster Hadoop avec Docker

Cours	Bases de données massives avancées (IFM30522)
Unité	UA1 — Projet 1
Groupe	G4 — Les_Hadoop_Riders
Image Docker	leshadoopriders/hadoop-tp-g4:1.0
Docker Hub	https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4
Code source	https://github.com/asinyopetro/Les_Hadoop_Riders_G4
Version Hadoop	3.3.6

Membres du groupe

Étudiant	Étudiant
Komla Petro Asinyo	Kassoum Dene
Joel Kazoni Tugirimana	Forbes Magène
Frank A Simo Ngounou	Wren Surprenant-Nicolson

Introduction

Ce rapport décrit le travail du groupe **G4 — Les_Hadoop_Riders** pour le Projet 1. On a déployé un cluster **Hadoop** avec **Docker** : stockage **HDFS**, jobs avec **YARN**, image publiée sur Docker Hub.

Le rapport reprend les trois parties de l'énoncé : théorie, manipulations pratiques, puis monitoring et problèmes rencontrés.

Lexique (sigles utiles)

Sigle / terme	Signification
HDFS	<i>Hadoop Distributed File System</i> — système de fichiers distribué
YARN	<i>Yet Another Resource Negotiator</i> — gestionnaire de ressources
3V	Volume, Vitesse, Variété (caractéristiques du Big Data)
NameNode	Nœud maître HDFS (namespace / métadonnées)
DataNode	Nœud de stockage des blocs HDFS
RM / NM	ResourceManager / NodeManager (YARN)
MapReduce	Modèle de calcul (phase Map puis Reduce)
Docker Hub	Registre public d'images Docker
UI	Interface web (NameNode :9870, YARN :8088)

Partie 1 — Contexte théorique et architecture (5 pts)

1. Cas d'usage Big Data (1.5 pt)

Nous avons choisi le secteur de l'**e-commerce** (commerce électronique).

Exemple concret : une chaîne de magasins qui conserve les tickets de caisse, les logs du site web et les mouvements de stock. Chaque jour le volume grossit, les données arrivent souvent, et les formats changent (CSV, JSON, textes, images produits). Une base relationnelle seule peut devenir insuffisante lorsque les historiques sont très longs et que les formats sont hétérogènes.

Les **3V** du Big Data dans ce contexte :

V	Signification	Application e-commerce
Volume	Quantité de données	Millions d'événements, historique sur plusieurs années
Vélocité	Vitesse d'arrivée	Commandes et paiements en continu ; réaction rapide (fraude, rupture)
Variété	Diversité des formats	Tables, logs, JSON, images produits

Dans ce scénario, **HDFS** permet de stocker ces données de façon distribuée sur plusieurs machines. **YARN** permet ensuite de lancer des traitements (MapReduce, etc.) directement sur le cluster, près des données.

2. HDFS vs système de fichiers classique (1.5 pt)

Critère	FS classique (NTFS, ext4...)	HDFS
Emplacement	Une seule machine	Plusieurs machines (cluster)
Organisation	Fichier entier sur un disque	Fichier découpé en blocs
Métadonnées	Gérées par l'OS local	Gérées par le NameNode
Tolérance aux pannes	Faible (disque unique)	Forte grâce à la réplication
Contenu des fichiers	Sur le même disque que les métadonnées	Sur les DataNodes

Sur un système local, un fichier vit sur **une** machine. Avec **HDFS**, le fichier est coupé en blocs (souvent 128 Mo), répartis et répliqués sur plusieurs DataNodes. Le NameNode conserve le namespace (noms, dossiers, emplacement des blocs) mais ne stocke pas le contenu des fichiers.

En pratique : le FS local est centralisé ; HDFS est distribué et plus tolérant aux pannes grâce à la réplication. Si un DataNode tombe, d'autres copies du bloc restent disponibles.

3. Rôle de YARN — ResourceManager et NodeManager (2 pts)

YARN (*Yet Another Resource Negotiator*) gère les ressources CPU/mémoire du cluster lorsqu'une application est soumise.

Composant	Où ?	Rôle
ResourceManager (RM)	Master	Reçoit la demande, connaît les ressources, alloue des conteneurs, démarre l'ApplicationMaster
NodeManager (NM)	Chaque worker	Démarre et surveille les conteneurs sur sa machine , remonte l'état au RM
ApplicationMaster	Conteneur alloué	« Chef » du job : suit l'exécution Map/Reduce

Enchaînement typique : le client soumet un job → le ResourceManager alloue des ressources → l'ApplicationMaster coordonne le job → les NodeManagers exécutent les tâches dans des conteneurs.

À retenir : le ResourceManager **orchestre** ; le NodeManager **exécute** localement.

Méthode de travail

Nous avons d'abord préparé l'image Docker et le fichier `docker-compose.yml` (1 master + 5 workers). Ensuite nous avons vérifié le cluster (`dfsadmin -report` , UI NameNode), puis enchaîné les commandes HDFS et le job YARN. Les captures d'écran du rapport ont été prises pendant ces tests. Enfin, l'image a été poussée sur Docker Hub et le code déposé sur GitHub.

Partie 2 — Déploiement et manipulation pratique (15 pts)

A. Mise en place de l'environnement Docker (4 pts)

1. Déploiement NameNode + 5 DataNodes (1.5 pt)

Nous avons déployé **six conteneurs** sur le réseau Docker `hadoop-net` , à partir d'une image unique dont le rôle (master ou worker) est choisi via la variable d'environnement `HADOOP_ROLE` .

Conteneur	Rôles
hadoop-master	NameNode, ResourceManager, SecondaryNameNode
hadoop-worker1 ... hadoop-worker5	DataNode + NodeManager

Ports exposés :

Port	Service
9870	Interface web HDFS (NameNode UI)
8088	Interface web YARN
9000	HDFS RPC

```
docker compose up --build -d
docker ps
```

La commande `up --build -d` construit l'image si nécessaire, crée le réseau et démarre les six services en arrière-plan. `docker ps` permet de vérifier que les conteneurs sont bien *Up*.

```
Windows PowerShell — Les_Hadoop_Riders_G4

PS C:\Users\asiny\Desktop\Les_Hadoop_Riders_G4> docker ps
NAME                STATUS              PORTS
hadoop-worker2      Up 10 hours         8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker3      Up 10 hours         8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker5      Up 10 hours         8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker1      Up 10 hours         8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-worker4      Up 10 hours         8042/tcp, 8088/tcp, 9000/tcp, 9864/tcp, 9870/tcp
hadoop-master       Up 10 hours         0.0.0.0:8088->8088/tcp, [::]:8088->8088/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 0.0.0.0:9870->9870/tcp, [::]:9870->9870/tcp

PS C:\Users\asiny\Desktop\Les_Hadoop_Riders_G4> docker exec -it hadoop-master hdfs dfsadmin -report
Configured Capacity: 5405505884160 (4.92 TB)
Present Capacity: 5032864239616 (4.58 TB)
DFS Remaining: 5032862023680 (4.58 TB)
DFS Used: 2215936 (2.11 MB)
DFS Used%: 0.00%
Replicated Blocks:
  Under replicated blocks: 0
  Blocks with corrupt replicas: 0
  Missing blocks: 0
  Missing blocks (with replication factor 1): 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
Erasure Coded Block Groups:
  Low redundancy block groups: 0
  Block groups with corrupt internal blocks: 0
  Missing block groups: 0
  Low redundancy blocks with highest priority to recover: 0
  Pending deletion blocks: 0
-----
Live datanodes (5):

Name: 172.23.0.3:9866 (hadoop-worker5.les_hadoop_riders_g4_hadoop-net)
Hostname: hadoop-worker5
Decommission Status : Normal
Configured Capacity: 1081101176832 (1006.85 GB)
```

Capture terminal — docker ps / dfsadmin

2. Vérification des DataNodes (1.5 pt)

```
docker exec -it hadoop-master hdfs dfsadmin -report
```

Cette commande interroge le NameNode et affiche l'état du cluster DFS (capacité, DataNodes vivants, etc.).

Résultat observé : Live datanodes (5) — le cluster est opérationnel.
Interface NameNode : <http://localhost:9870>

HadoopOverviewDatanodesDatanode Volume FailuresSnapshotStartup ProgressUtilities

Overview'hadoop-master:9000'(✓active)

Started:

Sat Oct 03 11:02:46 -0400 2026

Version:

3.3.6, r1be78238728da9266a4f88195058f08d012bf9c

Compiled:

Sun Jun 18 04:22:00 -0400 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)

Cluster ID:

CID-c8f2f423-c206-4cdf-9c10-5573319713da

Block Pool ID:

BP-329230766-172.23.0.2-1791033373712

Summary

Security is off.
Safemode is off.
32 files and directories, 9 blocks (9 replicated blocks, 0 erasure coded block groups) = 41 total filesystem object(s).
Heap Memory used 89.9 MB of 242 MB Heap Memory. Max Heap Memory is 2.4 GB.
Non Heap Memory used 69.58 MB of 72.25 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:

4.92 TB

Configured Remote Capacity:

0 B

DFS Used:

2.18 MB (0%)

Non DFS Used:

90.97 GB

DFS Remaining:

4.58 TB (93.11%)

Block Pool Used:

2.18 MB (0%)

DataNodes usages% (Min/Median/Max/stdDev):

0.00% / 0.00% / 0.00% / 0.00%

Live Nodes

5 (Decommissioned: 0, In Maintenance: 0)

Dead Nodes

0 (Decommissioned: 0, In Maintenance: 0)

Decommissioning Nodes

0

UI NameNode

HadoopOverviewDatanodesDatanode Volume FailuresSnapshotStartup ProgressUtilities

Datanode Information

✓ In service ⬇ Down ⬆ Decommissioning ⚠ Decommissioned ⛔ Decommissioned & dead
➡ Entering Maintenance ⚡ In Maintenance ⚡ In Maintenance & dead

Datanode usage histogram

5

0102030405060708090100

Disk usage of each DataNode (%)

In operation

DataNode State: All Show 25 entries Search:

Node	Http Address	Last contact	Last Block Report	Used	Non DFS Used	Capacity	Blocks	Block pool used	Version
✓ default-rack/hadoop-worker1:9866 (172.23.0.5:9866)	http://hadoop-worker1:9866	2s	250m	684 KB	18.19 GB	1006.85 GB	7	684 KB (0%)	3.3.6
✓ default-rack/hadoop-worker3:9866 (172.23.0.4:9866)	http://hadoop-worker3:9866	2s	354m	344.99 KB	18.19 GB	1006.85 GB	3	344.99 KB (0%)	3.3.6
✓ default-rack/hadoop-worker4:9866 (172.23.0.7:9866)	http://hadoop-worker4:9866	2s	263m	353.43 KB	18.19 GB	1006.85 GB	5	353.43 KB (0%)	3.3.6
✓ default-rack/hadoop-worker5:9866 (172.23.0.3:9866)	http://hadoop-worker5:9866	2s	276m	178.31 KB	18.19 GB	1006.85 GB	5	178.31 KB (0%)	3.3.6
✓ default-rack/hadoop-worker2:9866 (172.23.0.6:9866)	http://hadoop-worker2:9866	2s	278m	667.16 KB	18.19 GB	1006.85 GB	6	667.16 KB (0%)	3.3.6

Liste des DataNodes

3. Publication de l'image (1 pt)

```
docker login
docker push leshadoopriders/hadoop-tp-g4:1.0
```

URL publique : <https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4>

Note : Docker Hub impose les minuscules dans le nom de dépôt, d'où `g4` plutôt que `G4` .

B. Manipulation avancée sur HDFS (6 pts)

Pour coller au cas d'usage e-commerce, nous avons préparé un fichier local `data/transactions.csv` (colonnes ID, Date, Montant, Magasin — 10 lignes) et un second fichier `transactions_part2.csv` pour tester la fusion.

4. Répertoires et droits (2 pts)

```
hdfs dfs -mkdir -p /data/ventes/2026
hdfs dfs -chmod 755 /data/ventes/2026
hdfs dfs -put -f /data/transactions.csv /data/ventes/2026/
hdfs dfs -chmod 644 /data/ventes/2026/transactions.csv
```

Nous créons l'arborescence `/data/ventes/2026` , appliquons des droits sur le dossier (`755`), uploadons le CSV, puis fixons les droits du fichier (`644`).

5. Transfert et vérification des blocs (2 pts)

```
hdfs fsck /data/ventes/2026/transactions.csv -files -blocks -locations
hdfs dfs -stat "%n | taille=%b | replication=%r | block_size=%o"
/data/ventes/2026/transactions.csv
```

Indicateur	Valeur observée
Taille	377 octets
Réplication initiale	3
Nombre de blocs	1 (fichier petit ; bloc par défaut 128 Mo)
État FSCK	HEALTHY

Le fait d'avoir **un seul bloc** est normal : la taille du fichier est très inférieure à la taille de bloc HDFS (128 Mo). La réplication 3 signifie que ce bloc existe en trois copies sur le cluster.

```
root@hadoop-master:~# HDFS

root@hadoop-master:~# hdfs dfs -ls /data/ventes/2026
Found 2 items
-rw-r--r-- 2 hadoop supergroup 377 2026-10-03 13:30 /data/ventes/2026/transactions.csv
-rw-r--r-- 3 hadoop supergroup 567 2026-10-03 13:31 /data/ventes/2026/transactions_merged.csv
root@hadoop-master:~# hdfs dfs -cat /data/ventes/2026/transactions.csv | head -n 5
ID,Date,Montant,Magasin
T001,2026-01-05,49.99,Montreal-Centre
T002,2026-01-06,129.50,Laval
T003,2026-01-07,15.25,Quebec
T004,2026-01-08,250.00,Montreal-Ouest
root@hadoop-master:~# hdfs fsck /data/ventes/2026/transactions.csv -files -blocks -locations 2>&1 | head -n 25
Connecting to namenode via http://hadoop-master:9870/fsck?ugi=root&files=1&blocks=1&locations=1&path=%2Fdata%2Fventes%2F2026%2Ftransactions.csv
FSCK started by root (auth:SIMPLE) from /172.23.0.2 for path /data/ventes/2026/transactions.csv at Sun Oct 04 01:02:57 GMT 2026

/data/ventes/2026/transactions.csv 377 bytes, replicated: replication=2, 1 block(s): OK
0. BP-329230766-172.23.0.2-1791033373712:blk_1073741825_1001 len=377 Live_repl=2 [DatanodeInfoWithStorage[172.23.0.4:9866,DS-6b0bc5c8-0343-4441-977a-45186fdef003,DISK], DatanodeInfoWithStorage[172.23.0.6:9866,DS-f042a204-27cf-4970-b237-39c216ef47fb,DISK]]

Status: HEALTHY
Number of data-nodes: 5
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 377 B
Total files: 1
Total blocks (validated): 1 (avg. block size 377 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 3
Average block replication: 2.0
Missing blocks: 0
Corrupt blocks: 0
```

Capture terminal — HDFS

6. Lecture et concaténation (1 pt)

```
hdfs dfs -cat /data/ventes/2026/transactions.csv | head -n 5
hdfs dfs -put -f /data/transactions_part2.csv /data/ventes/2026/
hdfs dfs -getmerge /data/ventes/2026 /tmp/transactions_merged.csv
hdfs dfs -put -f /tmp/transactions_merged.csv
/data/ventes/2026/transactions_merged.csv
```

Les 5 premières lignes sont lues directement depuis HDFS. Les deux fichiers du dossier ont ensuite été fusionnés via `getmerge`, puis le résultat a été remis dans HDFS.

7. Suppression / corbeille (1 pt)

```
hdfs dfs -rm /data/ventes/2026/transactions_part2.csv
hdfs dfs -D fs.trash.interval=10080 -rm /data/ventes/2026/to_delete.csv
hdfs dfs -ls -R /user/hadoop/.Trash
# suppression définitive :
# hdfs dfs -rm -skipTrash /chemin/fichier
```

Sans `fs.trash.interval`, la suppression peut être définitive. Avec un intervalle > 0 , le fichier passe dans `.Trash`. L'option `-skipTrash` force la suppression définitive, utile pour nettoyer l'espace DFS.

C. Exécution d'un job YARN (5 pts)

Exemple choisi : **calcul de π** par méthode de Monte Carlo, via l'exemple officiel MapReduce fourni avec Hadoop. Cet exemple est pédagogique : il démarre rapidement et permet de vérifier que YARN alloue bien des ressources et que les NodeManagers exécutent les tâches.

```
yarn jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-3.3.6.jar pi 4 1000
```

Les paramètres **4** et **1000** correspondent à **4** tâches map et **1000** échantillons par map.

Champ	Résultat
Application	application_1791039798429_0001
Nom	QuasiMonteCarlo
État final	SUCCEEDED
Estimation	$\pi \approx$ 3.14
Paramètres	4 maps, 1000 samples

```
root@hadoop-master — YARN

root@hadoop-master:/# yarn application -list -appStates All
docker : 2026-10-04 01:03:01,304 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at
hadoop-master/172.23.0.2:8032
Au caractère C:\Users\asiny\AppData\Local\Temp\ps-script-67860ea5-b239-4dd4-997c-850c8a40b275.ps1:107 : 9
+ $yarn = docker exec hadoop-master bash -c 'yarn application -list -ap ...
+ ~~~~~
+ CategoryInfo          : NotSpecified: (2026-10-04 01:0...172.23.0.2:8032:String) [], RemoteException
+ FullyQualifiedErrorId : NativeCommandError

Total number of applications (application-types: [], states: [NEW, NEW_SAVING, SUBMITTED, ACCEPTED, RUNNING, FINISHED, FAILED, KILLED] and tags:
[]):1
Final-State      Application-Id      Application-Name      Application-Type      User      Queue      State
application_1791039798429_0001      QuasiMonteCarlo      MAPREDUCE      root      default      FINISHED
SUCCEEDED      100% http://hadoop-worker1:19888/jobhistory/job/job_1791039798429_0001
root@hadoop-master:/# yarn application -status application_1791039798429_0001 2>&1 | head -n 30
2026-10-04 01:03:05,625 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at hadoop-master/172.23.0.2:8032
2026-10-04 01:03:06,463 INFO conf.Configuration: resource-types.xml not found
2026-10-04 01:03:06,464 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
Application Report :
  Application-Id : application_1791039798429_0001
  Application-Name : QuasiMonteCarlo
  Application-Type : MAPREDUCE
  User : root
  Queue : default
  Application Priority : 0
  Start-Time : 1791070566554
  Finish-Time : 1791070642461
  Progress : 100%
  State : FINISHED
  Final-State : SUCCEEDED
  Tracking-URL : http://hadoop-worker1:19888/jobhistory/job/job_1791039798429_0001
  RPC Port : 41245
  AM Host : hadoop-worker1
  Aggregate Resource Allocation : 210745 MB-seconds, 245 vcore-seconds
  Aggregate Resource Preempted : 0 MB-seconds, 0 vcore-seconds
  Log Aggregation Status : DISABLED
```

Capture terminal — YARN



FINIS

Cluster

About
Nodes
Node Labels
Applications
NEW
NEW SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Cluster Metrics

Apps Submitted	0	Apps Pending	0	Apps Running	1	Apps Completed	0	Containers Running	0	Used R
----------------	---	--------------	---	--------------	---	----------------	---	--------------------	---	--------

Cluster Nodes Metrics

Active Nodes	0	Decommissioning Nodes	0	Decommissioned Nodes
--------------	---	-----------------------	---	----------------------

Scheduler Metrics

Scheduler Type	Capacity Scheduler	Scheduling Resource Type	[memory-mb (unit=Mi), vcores]	Minimum Allocation	<memory:256, vCores:1>
----------------	--------------------	--------------------------	-------------------------------	--------------------	------------------------

Show 20 entries

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime
application_1791039798429_0001	root	QuasiMonteCarlo	MAPREDUCE		default	0	Sat Oct 3 19:36:06 -0400 2026	Sat Oct 3 19:36:11 -0400 2026	Sat Oct 3 19:37:22 -0400 2026

Showing 1 to 1 of 1 entries

UI YARN — applications



Application application_1791039798429_0001

Logged in as: dr.who

Cluster

About
Nodes
Node Labels
Applications
NEW
NEW SAVING
SUBMITTED
ACCEPTED
RUNNING
FINISHED
FAILED
KILLED
Scheduler

Tools

Application Overview

User: root
Name: QuasiMonteCarlo
Application Type: MAPREDUCE
Application Tags:
Application Priority: 0 (Higher integer value indicates higher priority)
YarnApplicationState: FINISHED
Queue: default
FinalStatus Reported by AM: SUCCEEDED
Started: Sat Oct 03 23:36:06 +0000 2026
Launched: Sat Oct 03 23:36:11 +0000 2026
Finished: Sat Oct 03 23:37:22 +0000 2026
Elapsed: 1mins, 15sec
Tracking URL: History
Log Aggregation Status: DISABLED
Application Timeout (Remaining Time): Unlimited
Diagnostics:
Unmanaged Application: false
Application Node Label expression: <Not set>
AM container Node Label expression: <DEFAULT_PARTITION>

Application Metrics

Total Resource Preempted: <memory 0, vCores 0>
Total Number of Non-AM Containers Preempted: 0
Total Number of AM Containers Preempted: 0
Resource Preempted from Current Attempt: <memory 0, vCores 0>
Number of Non-AM Containers Preempted from Current Attempt: 0
Aggregate Resource Allocation: 210745 MB-seconds, 245 vcore-seconds
Aggregate Preempted Resource Allocation: 0 MB-seconds, 0 vcore-seconds

Show 20 entries

Search:

Attempt ID	Started	Node	Logs	Nodes blacklisted by the app	Nodes blacklisted by the system
appattempt_1791039798429_0001_000001	Sat Oct 3 19:36:06 -0400 2026	http://hadoop-worker1.8042	Logs	0	0

Showing 1 to 1 of 1 entries

First Previous 1 Next Last

Détail application SUCCEEDED

Partie 3 — Administration, monitoring et analyse (10 pts)

1. Interfaces web (4 pts)

Les interfaces web permettent de contrôler l'état du cluster sans rester uniquement en ligne de commande.

NameNode — <http://localhost:9870>

Le cluster affiche **5 DataNodes** actifs. L'espace DFS utilisé reste faible (environnement de laboratoire). Aucun *missing block* (bloc manquant) observé pendant nos tests. L'onglet DataNodes confirme que chaque worker est bien enregistré.

YARN — <http://localhost:8088>

L'application pi apparaît en état **FINISHED / SUCCEEDED**. Les métriques montrent l'allocation mémoire et le temps d'exécution (environ 198589 MB-secondes et 216 vcore-secondes sur un des runs). On peut aussi ouvrir le détail de l'application pour voir les tentatives Map/Reduce.

2. Facteur de réplication (3 pts)

```
hdfs dfs -setrep -w 2 /data/ventes/2026/transactions.csv
```

Après la commande, la réplication du fichier passe à **2**. Le NameNode met à jour la cible. Comme on diminuait de 3 à 2, une copie de bloc superflu est retirée sur les DataNodes. L'option `-w` (*wait*) attend la fin de l'opération avant de rendre la main.

Changer la réplication est utile pour équilibrer **fiabilité** (plus de copies) et **coût en espace disque** (moins de copies).

3. Retour d'expérience / troubleshooting (3 pts)

#	Problème	Cause	Solution
1	<code>pull access denied</code>	Compose téléchargeait une image pas encore sur Hub	<code>pull_policy: never</code> + build local
2	<code>repository name must be lowercase</code>	Majuscules dans le tag (<code>G4</code>)	Tag final <code>leshadoopridders/hadoop-tp-g4:1.0</code>
3	Build bloqué longtemps	Téléchargement trop lent depuis <code>archive.apache.org</code>	Base <code>apache/hadoop:3.3.6</code> + notre config / <code>entrypoint.sh</code>

Ces incidents nous ont permis de mieux comprendre le cycle de vie d'une image Docker (build local → test → push Hub) et les contraintes de nommage de Docker Hub.

Conclusion

Le groupe **Les_Hadoop_Riders** a déployé un cluster Hadoop **1 master + 5 workers** avec Docker, manipulé HDFS (droits, fsck, lecture, fusion, corbeille, setrep), exécuté un job MapReduce π terminé en **SUCCEEDED**, et publié l'image `leshadoopridders/hadoop-tp-g4:1.0` sur Docker Hub.

Ce travail montre qu'un environnement Big Data pédagogique peut être reproduit de façon reproductible grâce à la conteneurisation, tout en couvrant les notions centrales du cours : **3V**, **HDFS**, **YARN**, monitoring et administration de base.

Annexes

A. Structure du code source

```
Les_Hadoop_Riders_G4/
├── Dockerfile
├── docker-compose.yml
├── entrypoint.sh
├── config/
│   ├── core-site.xml
│   ├── hdfs-site.xml
│   ├── yarn-site.xml
│   ├── mapred-site.xml
│   └── workers
├── data/
│   ├── transactions.csv
│   └── transactions_part2.csv
├── scripts/
│   ├── demo-hdfs.sh
│   └── demo-yarn-pi.sh
├── captures/
└── README.md
```

B. Liste des captures

Fichier	Contenu
screenshot-terminal-01.png	<code>docker ps</code> / dfsadmin
screenshot-terminal-02.png	Manipulations HDFS
screenshot-terminal-03.png	Sortie job π
screenshot-namenode.png	UI NameNode
screenshot-datanodes.png	Liste des 5 DataNodes
screenshot-yarn.png	UI YARN — applications
screenshot-yarn-app.png	Détail application SUCCEEDED

C. Liens du projet

Ressource	URL
Docker Hub	https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4
GitHub	https://github.com/asinyopetro/Les_Hadoop_Riders_G4
UI NameNode (local)	http://localhost:9870
UI YARN (local)	http://localhost:8088